

Problem Set 04

Ejercicio 1

Parte1

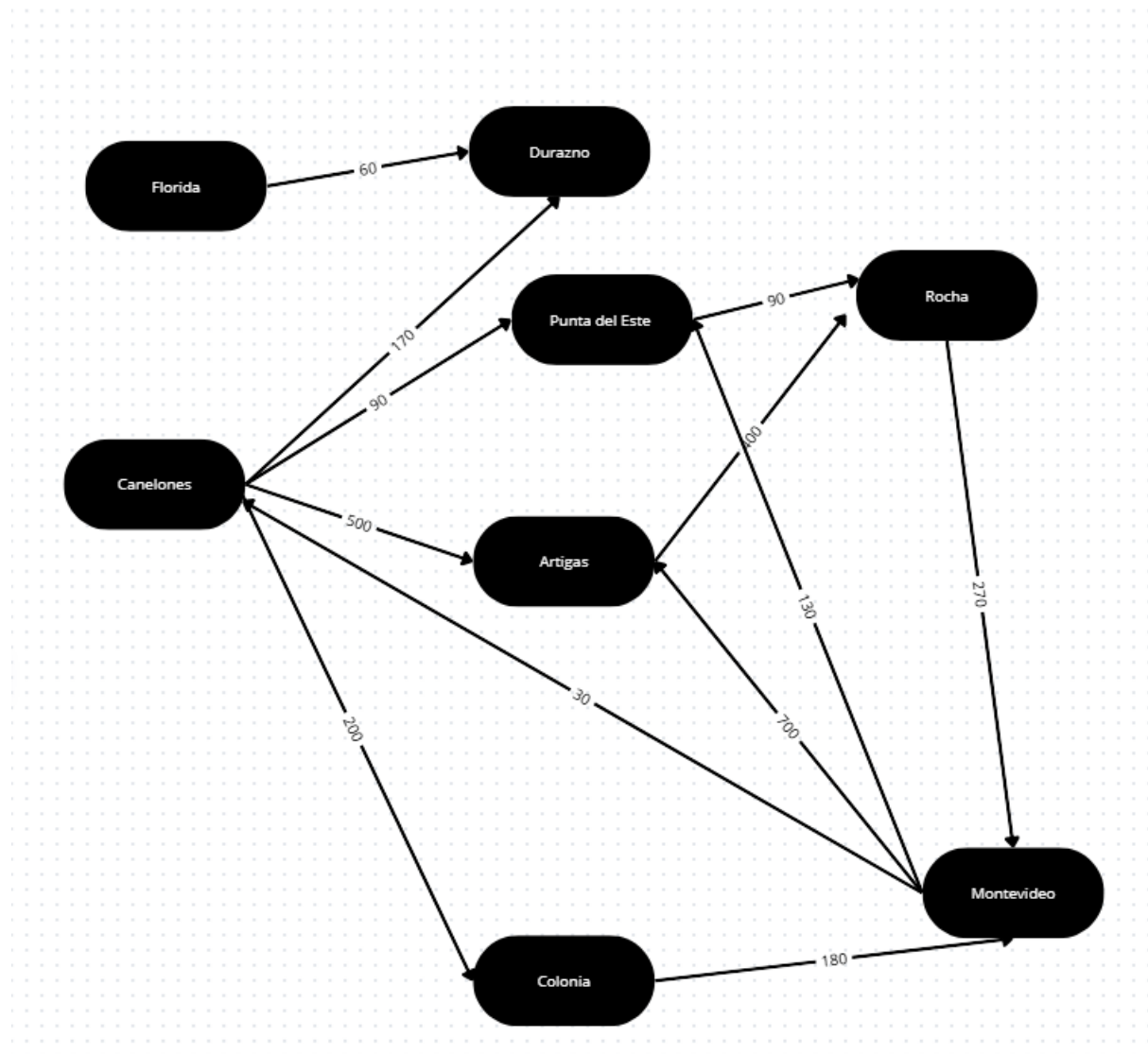
1.a)

	Artigas	Canelones	Colonia	Durazno	Florida	Montevideo	Punta del Este	Rocha
Artigas	0	∞	∞	∞	∞	∞	∞	400
Canelones	500	0	200	170	∞	∞	90	∞
Colonia	∞	∞	0	∞	∞	180	∞	∞
Durazno	∞	∞	∞	0	∞	∞	∞	∞
Florida	∞	∞	∞	60	0	∞	∞	∞
Montevideo	700	30	∞	∞	∞	0	130	∞
Punta del Este	∞	∞	∞	∞	∞	∞	0	90
Rocha	∞	∞	∞	∞	∞	270	∞	0

1.b)

Artigas:		-> Rocha (400)						
Canelones:		-> Artigas (500)		-> Colonia (200)		-> Durazno (170)		-> Punta del Este (90)
Colonia:		-> Montevideo (180)						
Florida:		-> Durazno (60)						
Montevideo:		-> Artigas (700)		-> Canelones (30)		-> Punta del Este (130)		
Punta del Este:		-> Rocha (90)						
Rocha:		-> Montevideo (270)						
Durazno:		(sin adyacentes)						

2.



Parte 2

1						
2	dijkstra (origen, costo, v)	Primera iteración	Segunda iteración	Tercera iteración	Cuarta iteración	Quinta iteración
3	#S-> nodos visitados #D-> vector de costos desde origen #V-> vertices del grafo					
4	S<- {Origen}					
5	D=[]	S<- {Montevideo} D={Montevideo}=0, D{Artigas}=700, D{Canelones}=30, D{Colonia}=∞, D{Durazno}=∞, D{PDE}=130, D{Rocha}=∞, D{Florida}=∞				
6						
7	Para todo i en V hacer D[i] = costo (origen, i)					
8	Mientras V<>S hacer					
9	Elegir w perteneciente a V-S, tal que D[w]	w = Canelones	w = PDE	w = Durazno	w = Rocha	w = Colonia
10	Agregar w a S	S = {Montevideo, Canelones}	S = {Montevideo, Canelones, PDE}	S = {Montevideo, Canelones, PDE, Durazno}	S = {Montevideo, Canelones, PDE, Durazno, Rocha}	S = {Montevideo, Canelones, PDE, Durazno, Rocha, Colonia}
11	Para todo v en V-S hacer	D = {D{Montevideo}=0, D{Artigas}=700, D{Canelones}=30, D{Colonia}=∞, D{Durazno}=∞, D{PDE}=130, D{Rocha}=∞, D{Florida}=∞}, {D{Canelones}=0, D{Durazno}=170, D{PDE}=90, D{Artigas}=500, D{Colonia}=200}, D{Artigas}=min(700, 30+500)=530, D{Colonia}=min(∞, 30+200)=230, D{Durazno}=min(∞, 30+170)=200, D{PDE}=min(130, 30+90)=120	D = {D{Montevideo}=0, D{Artigas}=530, D{Canelones}=30, D{Colonia}=230, D{Durazno}=200, D{PDE}=120, D{Rocha}=∞, D{Florida}=∞}, {D{PDE}=0, D{Rocha}=90}, D{Rocha}=min(∞, 30+90+90)=210	D = {D{Montevideo}=0, D{Artigas}=530, D{Canelones}=30, D{Colonia}=230, D{Durazno}=200, D{PDE}=120, D{Rocha}=210, D{Florida}=∞}, {D{Durazno}=0, no tiene adyacentes } {D{Rocha}=0, D{Montevideo}=270}, D{Montevideo}=min(0, 30+90+90+270)=0	D = {D{Montevideo}=0, D{Artigas}=530, D{Canelones}=30, D{Colonia}=230, D{Durazno}=200, D{PDE}=120, D{Rocha}=210, D{Florida}=∞}, {D{Colonia}=0, D{Montevideo}=180}, D{Montevideo}=min(0, 230+180)=0	D = {D{Montevideo}=0, D{Artigas}=530, D{Canelones}=30, D{Colonia}=230, D{Durazno}=200, D{PDE}=120, D{Rocha}=210, D{Florida}=∞}, {D{Artigas}=0, D{Rocha}=400}, D{Rocha}=min(210, 530+400)=210
12	Fin Par					
13	Fin mientras					
14	Fin mientras					
15	Fin mientras					
16						

Al aplicar el algoritmo de Dijkstra tomando como origen el aeropuerto de Montevideo, se obtienen los caminos mínimos hacia los demás aeropuertos.

Primero se inicializan los costos desde Montevideo. Los vértices directamente alcanzables son Canelones con costo 30, Punta del Este con costo 130 y Artigas con costo 700. Los demás vértices comienzan con costo infinito.

Luego, el algoritmo va seleccionando en cada paso el vértice no visitado con menor costo acumulado y actualizando los costos de sus adyacentes. El orden de selección es:

Montevideo → Canelones → Punta del Este → Durazno → Rocha → Colonia → Artigas

Florida no queda alcanzable desde Montevideo, porque su costo permanece infinito.

Destino	Costo mínimo desde Montevideo	Trayecto recorrido
Canelones	30	Montevideo → Canelones
Punta del Este	120	Montevideo → Canelones → Punta del Este
Durazno	200	Montevideo → Canelones → Durazno
Rocha	210	Montevideo → Canelones → Punta del Este → Rocha
Colonia	230	Montevideo → Canelones → Colonia

Artigas	530	Montevideo → Canelones → Artigas
Florida	∞	No existe camino desde Montevideo

Parte 3:

Para implementar la interfaz `IDirectedGraph`, optamos por una representación basada en listas de adyacencia en lugar de una matriz de adyacencia. Esta decisión nos permite almacenar únicamente las conexiones existentes de cada vértice, sin reservar espacio para todas las combinaciones posibles entre vértices.

Para esto utilizamos la API de colecciones de Java, especialmente `Map` y `List`. Una posible representación es:

`Map<V, List<Edge<V, D>>> adyacencias;`

En esta estructura, cada vértice se asocia con una lista de sus aristas salientes. También podría usarse una variante con mapas anidados:

`Map<V, Map<V, Edge<V, D>>> adyacencias;`

En ese caso, el primer `Map` permite acceder al vértice origen y el segundo `Map` permite acceder directamente a una arista usando como clave el vértice destino. Esta alternativa es más eficiente si se necesita consultar seguido si existe una arista específica entre dos vértices.

La principal diferencia frente a una matriz de adyacencia está en el uso de memoria. Una matriz necesita $O(V^2)$ espacio, porque reserva una posición para cada par posible de vértices. En cambio, una lista de adyacencia necesita $O(V + E)$, porque guarda los vértices y solamente las aristas existentes.

En la siguiente tabla se pueden apreciar los costos de memoria y tiempo de ejecución utilizando las distintas representaciones.

Operación	Matriz de adyacencia	Lista de adyacencia
Espacio utilizado	$O(V^2)$	$O(V+E)$
Agregar vértice	Costoso si hay que redimensionar	$O(1)$ promedio con <code>HashMap</code>
Agregar arista	$O(1)$	$O(1)$ promedio
Verificar si existe arista $u \rightarrow v$	$O(1)$	$O(\text{grado de salida})$ con <code>List</code> , o $O(1)$ promedio con un segundo <code>Map</code>

Obtener adyacentes de un vértice	$O(V)$	$O(\text{grado de salida})$
Recorrer todas las aristas	$O(V^2)$	$O(V+E)$

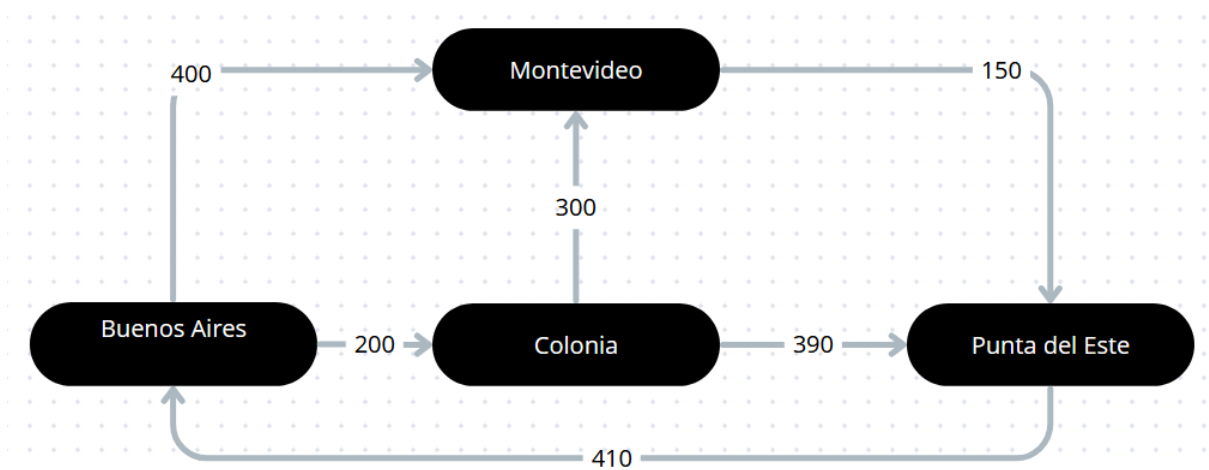
La matriz tiene la ventaja de permitir consultar en $O(1)$ si existe una arista entre dos vértices, pero paga ese beneficio usando más memoria y obligando a recorrer posiciones que pueden no representar aristas reales. En cambio, las listas de adyacencia permiten recorrer directamente los vecinos de cada vértice, lo cual se adapta mejor a algoritmos como búsqueda por profundidad (DFS), búsqueda por amplitud (BFS), Dijkstra y recorridos generales del grafo.

Respecto a las implementaciones concretas, se puede usar HashMap si no importa el orden de recorrido, ya que sus operaciones principales tienen costo promedio $O(1)$. Si se quiere mantener un orden al recorrer o mostrar los vértices, puede usarse LinkedHashMap. Si se necesitan vértices ordenados por clave, podría usarse TreeMap, aunque sus operaciones pasan a costar $O(\log V)$.

Para guardar las aristas salientes, ArrayList es una opción razonable porque permite recorrer los adyacentes de forma eficiente. Por estas razones, la representación con listas de adyacencia usando colecciones de Java resulta más flexible y eficiente en memoria que una matriz de adyacencia.

Ejercicio 2

Parte 1



2.

El algoritmo de Floyd permite calcular los caminos mínimos entre todos los pares de vértices de un grafo dirigido ponderado. Su estructura tiene tres ciclos anidados: uno para elegir el vértice intermedio k , otro para el origen i y otro para el destino j .

Por eso, si V es la cantidad de vértices, su orden temporal es $O(V^3)$. Además, utiliza una matriz de costos de tamaño $V \times V$, por lo que su consumo espacial es $O(V^2)$. Si se quiere recuperar el camino, también se puede usar una matriz auxiliar de predecesores o intermedios, que mantiene el mismo orden espacial.

En la versión clásica, el orden de Floyd no se reduce si se quieren obtener los caminos mínimos entre todos los pares de vértices. Si solo se necesitara calcular caminos desde un único origen, sería más conveniente usar Dijkstra. Sin embargo, en este caso se necesitan costos mínimos entre cualquier par de ciudades, por lo que Floyd resulta adecuado.

En operaciones reales, el costo $O(V^3)$ puede ser alto si el grafo tiene muchos vértices. Pero para este contexto, donde la cantidad de ciudades es chica, el impacto es bajo. La ventaja es que, una vez calculada la matriz final, se pueden consultar rápidamente los costos mínimos entre cualquier par de ciudades.

Parte 2

La matriz inicial de costos es:

Origen/Destino	Montevideo	Colonia	Buenos Aires	Punta del Este
Montevideo	0	∞	∞	150
Colonia	300	0	∞	390
Buenos Aires	400	200	0	∞
Punta del Este	∞	∞	410	0

Al aplicar Floyd, la matriz final de costos mínimos queda:

Origen/Destino	Montevideo	Colonia	Buenos Aires	Punta del Este
Montevideo	0	760	560	150
Colonia	300	0	800	390
Buenos Aires	400	200	0	550
Punta del Este	810	610	410	0

Tabla de recuperación de caminos:

En esta tabla, cada celda indica el vértice intermedio utilizado para recuperar el camino mínimo. El guion indica que el camino es directo y no requiere vértices intermedios.

Origen/Destino	Montevideo	Colonia	Buenos Aires	Punta del Este
Montevideo	0	Punta del Este	Punta del Este	-
Colonia	-	0	Punta del Este	-
Buenos Aires	-	-	0	Montevideo
Punta del Este	Buenos Aires	Buenos Aires	-	0

Tabla de excentricidad

Ciudad	Distancias hacia las demás	Excentricidad
Montevideo	760, 560, 150	760
Colonia	300, 800, 390	800
Buenos Aires	400, 200, 550	550
Punta del Este	810, 610, 410	810

El vértice con menor excentricidad corresponde al centro del grafo, por lo que debe elegirse ubicación del taller. Esto se debe a que la excentricidad de un vértice corresponde a la mayor distancia mínima desde ese vértice hacia todos los demás.

El taller se debe colocar en Buenos Aires, ya que es el vértice con menor excentricidad.

Parte 3

Lenguaje natural

Para recuperar los caminos mínimos obtenidos con Floyd, se utiliza una matriz auxiliar P. En cada posición $P[i][j]$ se guarda el vértice intermedio que permitió mejorar el camino entre i y j.

Si $P[i][j]$ vale 0 o null, significa que el camino mínimo entre esos vértices es directo. Si hay un vértice intermedio k, entonces la ruta se reconstruye dividiendo el camino en dos partes: primero desde i hasta k, y luego desde k hasta j.

Este proceso se aplica de forma recursiva hasta obtener todos los vértices del camino mínimo en el orden correcto.

Precondiciones

La matriz de costos mínimos ya fue calculada mediante el algoritmo de Floyd.

Existe una matriz auxiliar P para recuperar los caminos.

El vértice origen y el vértice destino pertenecen al grafo.

Si no existe camino entre el origen y el destino, la distancia correspondiente en la matriz de costos es infinito.

Postcondiciones

Si existe un camino entre el origen y el destino, se devuelve la lista de vértices que forman el camino mínimo.

Si el camino es directo, la ruta contiene únicamente el origen y el destino.

Si el camino tiene vértices intermedios, estos aparecen en el orden correcto.

Si no existe camino entre el origen y el destino, se devuelve una lista vacía.

Pseudocódigo

recuperarCamino(origen, destino, matrizCostos, matrizP)

```
    si matrizCostos[origen][destino] = infinito entonces
        devolver lista vacía
    fin si
    camino <- lista vacía
    agregar origen a camino
    agregarIntermedios(origen, destino, matrizP, camino)
    si origen != destino entonces
        agregar destino a camino
    fin si
```

```
    devolver camino
```

Fin

agregarIntermedios(origen, destino, matrizP, camino)

```
    k <- matrizP[origen][destino]
    si k = 0 o k = null entonces
        salir
    fin si
    agregarIntermedios(origen, k, matrizP, camino)
    agregar k a camino
    agregarIntermedios(k, destino, matrizP, camino)
```

fin

Ejercicio 5:

Parte 1:

Algoritmo en lenguaje natural

El requerimiento es listar todos los itinerarios posibles entre una ciudad origen y una ciudad destino. Esto equivale a encontrar todos los caminos simples (sin repetir vértices) entre origen y destino en un grafo dirigido ponderado, junto con el costo total de cada uno.

El algoritmo es un recorrido en profundidad implementado de forma iterativa con una pila. Funciona así:

- Se usan dos pilas en paralelo: una guarda los caminos parciales y la otra el costo acumulado de cada camino.
- Se empieza apilando el camino que contiene solo el origen, con costo 0.
- Mientras la pila no esté vacía, se saca un camino parcial y su costo.
- Si el último vértice del camino es el destino, ese camino se guarda como una solución.
- Si no es el destino, se mira cada arista saliente del último vértice. Por cada vecino que todavía no forma parte del camino (para evitar repetir vértices y entrar en ciclos), se crea un nuevo camino extendido y se apila junto con el costo actualizado.
- El algoritmo termina cuando la pila queda vacía. La lista de caminos guardados es el resultado.

Precondiciones: el grafo no es nulo y los vértices origen y destino pertenecen al grafo.

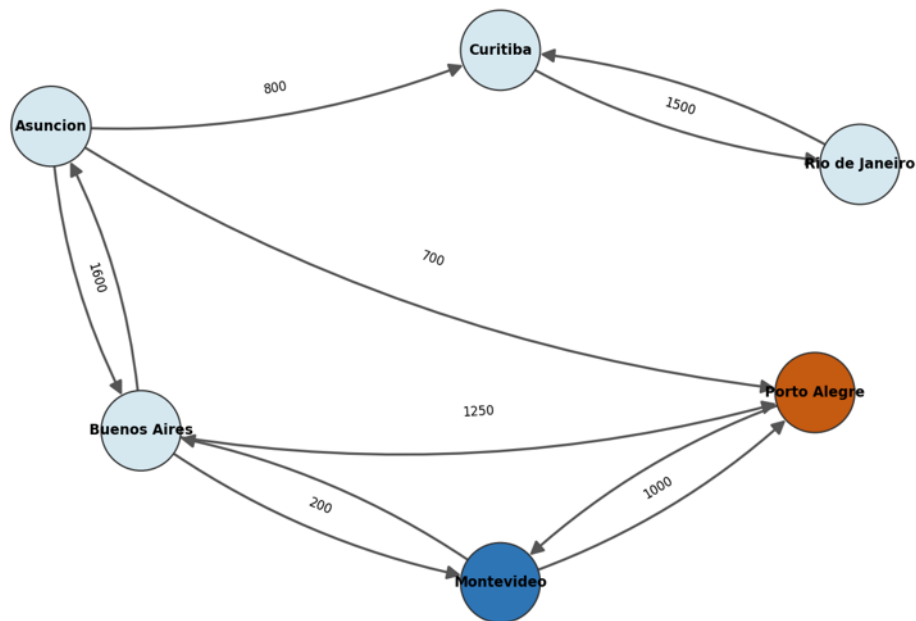
Postcondición: se obtiene la lista de todos los caminos simples entre origen y destino, cada uno con sus vértices y su costo total. Si no existe ninguno, la lista es vacía.

Grafo utilizado

Se utiliza el grafo de la aerolínea del Ejercicio 4 (Cuadro 3), único que contiene Montevideo y Porto Alegre con aristas dirigidas ponderadas. Su lista de adyacencias es:

Vértice	Aristas salientes (destino: costo)
Asunción	Buenos Aires: 1600, Curitiba: 800, Porto Alegre: 700
Buenos Aires	Asunción: 1600, Montevideo: 200, Porto Alegre: 1250
Curitiba	Rio de Janeiro: 1500
Montevideo	Buenos Aires: 200, Porto Alegre: 1000
Porto Alegre	Montevideo: 1000
Rio de Janeiro	Curitiba: 1500

Representación gráfica:



Paso a paso: Montevideo → Porto Alegre

La pila es LIFO: el último camino apilado es el primero en salir. En cada paso se muestra el camino que se saca de la pila, su costo y la acción resultante.

Paso	Camino extraído de la pila	Costo	Acción
1	[Montevideo]	0	No es destino. Se apilan [Mvd, Buenos Aires] (200) y [Mvd, Porto Alegre] (1000)
2	[Mvd, Porto Alegre]	1000	Es destino → Camino 1
3	[Mvd, Buenos Aires]	200	No es destino. Se apilan [Mvd, BA, Asunción] (1800) y [Mvd, BA, Porto Alegre] (1450)
4	[Mvd, BA, Porto Alegre]	1450	Es destino → Camino 2
5	[Mvd, BA, Asunción]	1800	No es destino. Se apilan [Mvd, BA, Asu, Curitiba] (2600) y [Mvd, BA, Asu, Porto Alegre] (2500)
6	[Mvd, BA, Asu, Porto Alegre]	2500	Es destino → Camino 3
7	[Mvd, BA, Asu, Curitiba]	2600	No es destino. Se apila [Mvd, BA, Asu, Cur, Rio de Janeiro] (4100)
8	[Mvd, BA, Asu, Cur, Rio de Janeiro]	4100	No es destino. Su único vecino (Curitiba) ya está en el camino → no se apila nada
9	Pila vacía	/	Fin del algoritmo. Se encontraron 3 caminos

Resultado de todas las conexiones Montevideo → Porto Alegre

#	Itinerario	Costo total (km)
1	Montevideo → Porto Alegre	1000
2	Montevideo → Buenos Aires → Porto Alegre	1450
3	Montevideo → Buenos Aires → Asunción → Porto Alegre	2500

Ejercicio 6:

Parte 1:

La forma más sencilla de detectar ciclos en un grafo dirigido es mediante una búsqueda en profundidad (**DFS**).

Se debe:

Recorrer cada vértice del grafo.

Cuando se visita un vértice, se lo marca como visitado o “en la pila de recursión” (camino actual de la búsqueda en profundidad).

Para cada adyacente: Si no fue visitado, continuar la búsqueda recursivamente. Si ya está en la pila de recursión, significa que encontramos un camino que vuelve a un vértice que todavía no terminó de procesarse, es decir, un ciclo.

Cuando terminamos de explorar un vértice, lo quitamos de la pila de recursión.

Si termina toda la exploración sin encontrar ese caso, el grafo es acíclico.

Pseudocódigo:

funcion tieneCiclos(grafo):

visitados $\leftarrow$ conjunto vacío

enRecursion $\leftarrow$ conjunto vacío

para cada vertice v del grafo:

si v no está visitado:

si dfsCiclos(v):

retornar true

retornar false

funcion dfsCiclos(v):

marcar v como visitado

agregar v a enRecursion

para cada adyacente w de v:

si w no está visitado:

si dfsCiclos(w):

retornar true

sino si w está en enRecursion:

retornar true

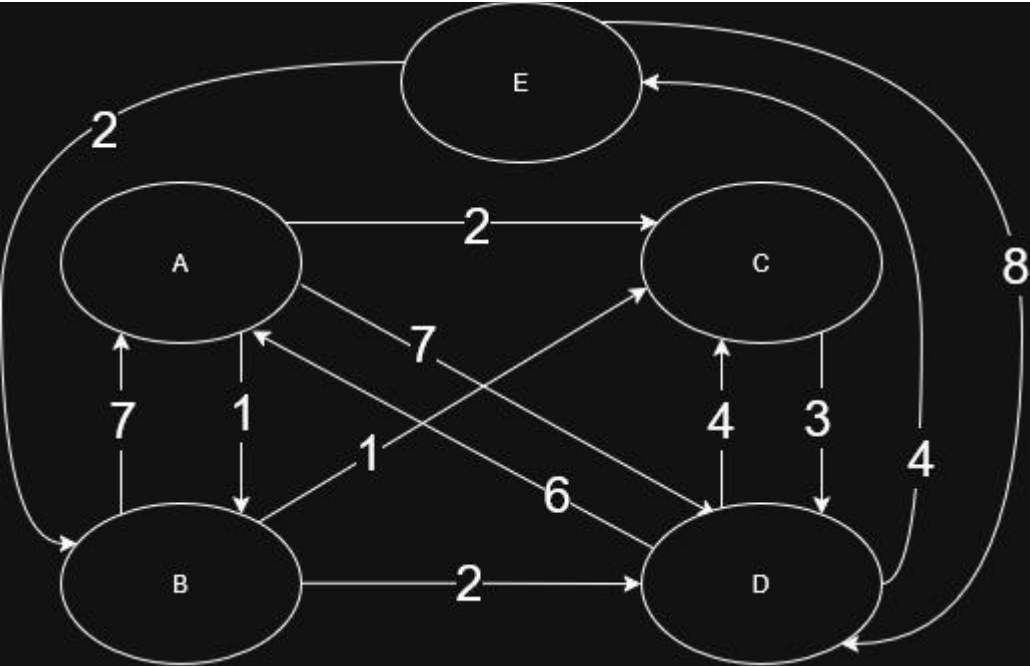
quitar v de enRecursion

retornar false

Ejercicio 7:

Parte 1:

1.



2.

Paso	Elegido	Visitados	Distancia / Predecesor				
			A	B	C	D	E
Inicial	B	{B}	7 / B	0 / -	1 / B	2 / B	∞ / -
1	C	{B, C}	7 / B	0 / -	1 / B	2 / B	∞ / -
2	D	{B, C, D}	7 / B	0 / -	1 / B	2 / B	6 / D
3	E	{B, C, D, E}	7 / B	0 / -	1 / B	2 / B	6 / D
4	A	{B, C, D, E, A}	7 / B	0 / -	1 / B	2 / B	6 / D

3.

	A	B	C	D	E
Predecesor	B	-	B	B	D

4.

Lenguaje Natural:

El algoritmo `imprimirCamino` recibe una estructura llamada `predecesor`, un nodo origen y un nodo destino.

Su objetivo es reconstruir el camino desde el nodo origen hasta el nodo destino usando la información guardada en `predecesor`.

Primero crea una lista vacía llamada `camino`. Luego comienza desde el nodo destino y va retrocediendo por sus predecesores. En cada paso, agrega el nodo actual al inicio de la lista `camino`.

Si en algún momento llega al nodo origen, significa que existe un camino entre el origen y el destino. Entonces imprime el camino completo y termina el algoritmo.

Si el algoritmo llega a un valor `null`, significa que no pudo llegar al origen siguiendo los predecesores. En ese caso, imprime un mensaje indicando que no existe camino entre el origen y el destino.

Precondiciones:

- Debe existir una estructura `predecesor` válida.
- `origen` y `destino` deben ser nodos válidos del grafo o estructura utilizada.
- La estructura `predecesor` debe indicar, para cada nodo alcanzable, cuál es su nodo anterior en el camino.
- Si un nodo no tiene predecesor, su valor en `predecesor` debe ser `null`.
- El arreglo o diccionario `predecesor` debe poder consultarse usando `predecesor[actual]`.

Postcondiciones:

- Si existe camino entre origen y destino se imprime la lista de nodos que forman el camino desde origen hasta destino.
- Si no existe camino entre origen y destino se imprime un mensaje indicando que no hay camino.
- El algoritmo no modifica la estructura `predecesor`. Solo construye una lista auxiliar llamada `camino`.

Seudocódigo:

Algoritmo `imprimirCamino`(`predecesor`, `origen`, `destino`)

`camino` ← lista vacía

actual $\leftarrow$ destino

Mientras actual $\neq$ null hacer

 agregar actual al inicio de camino

 Si actual = origen entonces

 imprimir camino

 retornar

 Fin Si

 actual $\leftarrow$ predecesor[actual]

Fin Mientras

 imprimir "No existe camino entre " + origen + " y " + destino

Fin Algoritmo

Parte 2:

1.

Vértice	Adyacentes
A	C(1), D(4)
B	A(6), E(3)
C	B(2), E(1)
D	C(5)
E	A(3)

2.

Costos

Origen	A	B	C	D	E
A	0	∞	1	4	∞
B	6	0	∞	∞	3
C	∞	2	0	∞	1
D	∞	∞	5	0	∞
E	3	∞	∞	∞	0

Origen	A	B	C	D	E
A	0	∞	1	4	∞
B	6	0	7	10	3
C	∞	2	0	∞	1
D	∞	∞	5	0	∞
E	3	∞	4	7	0

Origen	A	B	C	D	E
A	0	∞	1	4	∞
B	6	0	7	10	3
C	8	2	0	12	1
D	∞	∞	5	0	∞
E	3	∞	4	7	0

Origen	A	B	C	D	E
A	0	3	1	4	2
B	6	0	7	10	3
C	8	2	0	12	1
D	13	7	5	0	6
E	3	6	4	7	0

Origen	A	B	C	D	E
A	0	3	1	4	2
B	6	0	7	10	3
C	8	2	0	12	1
D	13	7	5	0	6
E	3	6	4	7	0

Origen	A	B	C	D	E
A	0	3	1	4	2
B	6	0	7	10	3
C	4	2	0	8	1
D	9	7	5	0	6
E	3	6	4	7	0

3.

Predecesores					
Origen	A	B	C	D	E
A	-	-	A	A	-
B	B	-	-	-	B
C	-	C	-	-	C
D	-	-	D	-	-
E	E	-	-	-	-

Origen	A	B	C	D	E
A	-	-	A	A	-
B	B	-	A	A	B
C	-	C	-	-	C
D	-	-	D	-	-
E	E	-	A	A	-

Origen	A	B	C	D	E
A	-	-	A	A	-
B	B	-	A	A	B
C	B	C	-	B	C
D	-	-	D	-	-
E	E	-	A	A	-

Origen	A	B	C	D	E
A	-	C	A	A	C
B	B	-	A	A	B
C	B	C	-	B	C
D	C	C	D	-	C
E	E	C	A	A	-

Origen	A	B	C	D	E
A	-	C	A	A	C
B	B	-	A	A	B
C	B	C	-	B	C
D	C	C	D	-	C
E	E	C	A	A	-

Lenguaje natural:

El algoritmo recuperarCamino recibe una estructura llamada predecesores, un nodo origen y un nodo destino.

Su objetivo es reconstruir el camino desde origen hasta destino, usando la matriz o tabla de predecesores predecesores[origen][actual].

Primero crea una lista vacía llamada camino. Luego comienza desde el nodo destino y va retrocediendo por sus predecesores hasta llegar al nodo origen.

En cada paso, agrega el nodo actual al inicio de la lista camino. Después actualiza actual con el nodo predecesor correspondiente.

Si durante ese proceso actual llega a null, significa que no existe camino entre el origen y el destino. En ese caso, imprime "No existe camino" y termina el algoritmo.

Si logra llegar al nodo origen, agrega el origen al inicio del camino e imprime la lista completa.

Precondiciones:

- Debe existir una estructura válida llamada predecesores.
- origen y destino deben ser nodos válidos del grafo.
- La estructura predecesores debe permitir consultas del tipo:
- `predecesores[origen][actual]`
- Para cada par de nodos conectados, `predecesores[origen][actual]` debe indicar el nodo anterior a actual en el camino desde origen.
- Si no existe camino hacia un nodo, su predecesor debe ser null.

Postcondiciones

- Después de ejecutar el algoritmo, ocurre una de estas dos cosas:
- Si existe camino desde origen hasta destino se imprime una lista con los nodos del camino en orden correcto, desde el origen hasta el destino.
- Si no existe camino desde origen hasta destino se imprime No existe camino
- El algoritmo no modifica la estructura predecesores. Solo crea y usa una lista auxiliar llamada camino.

Algoritmo recuperarCamino(predecesores, origen, destino)

camino $\leftarrow$ lista vacía

actual $\leftarrow$ destino

Mientras actual $\neq$ origen hacer

 agregar actual al inicio de camino

 actual $\leftarrow$ predecesores [origen][actual]

 Si actual = null entonces

 imprimir "No existe camino"

 terminar

 Fin Si

Fin Mientras

agregar origen al inicio de camino

imprimir camino

Fin Algoritmo

5.

Vértice	Excentricidad
A	4
B	10
C	8
D	9
E	7

6.

Vértice	Excentricidad	
A	4	Centro
B	10	
C	8	
D	9	
E	7	

Ejercicio 8:

Parte 1:

1. Para resolver este problema se puede modelar la red de zonas como un grafo no dirigido, ponderado y conexo.

Cada zona se representa como un vértice del grafo, y cada posible camino entre dos zonas se representa como una arista. El peso de cada arista indica la distancia o costo de construcción del camino entre esas dos zonas.

Como el objetivo es conectar todas las zonas utilizando caminos de costo mínimo, el problema se relaciona con el concepto de árbol de expansión mínimo. Un árbol de

expansión mínimo es un subgrafo que conecta todos los vértices, no tiene ciclos y tiene el menor costo total posible.

Para resolverlo se puede aplicar el algoritmo de Prim. Este algoritmo comienza desde un vértice origen y va agregando, en cada paso, la arista de menor peso que conecte un vértice ya visitado con un vértice todavía no visitado. De esta manera se construye progresivamente una solución que conecta todas las zonas con costo mínimo.

La representación elegida es una lista de adyacencias, ya que permite guardar para cada zona solamente sus conexiones reales con otras zonas, junto con sus pesos. Esto resulta más eficiente en memoria que una matriz de adyacencia cuando no existen conexiones entre todos los pares de vértices.

2.

Lista de adyacencias

Zona1:

- Zona2: 1
- Zona3: 5
- Zona4: 4
- Zona5: 7
- Zona6: 2

Zona2:

- Zona1: 1
- Zona3: 1
- Zona4: 3
- Zona6: 6

Zona3:

- Zona1: 5
- Zona2: 1
- Zona4: 2
- Zona5: 3
- Zona6: 5

Zona4:

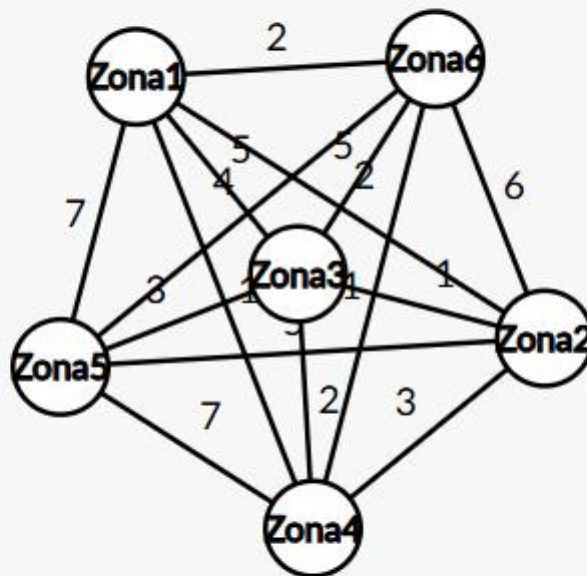
- Zona1: 4
- Zona2: 3
- Zona3: 2
- Zona5: 7
- Zona6: 2

Zona5:

- Zona1: 7
- Zona3: 3
- Zona4: 7

Zona6:

- Zona1: 2
- Zona2: 6
- Zona3: 5
- Zona4: 2



3.

4.

Paso	Visitados antes de elegir	Aristas candidatas principales	Arista elegida	Peso	Costo Acumulado
0	{Zona1}	-	-	-	0
1	{Zona1}	Z1-Z2(1), Z1-Z6(2), Z1-Z4(4), Z1-Z3(5), Z1-Z5(7)	Zona1 - Zona2	1	1
2	{Zona1, Zona2}	Z2-Z3(1), Z2-Z5(1), Z1-Z6(2), Z2-Z4(3), Z1-Z4(4), Z1-Z3(5), Z2-Z6(6), Z1-Z5(7)	Zona2 - Zona3	1	2
3	{Zona1, Zona2, Zona3}	Z2-Z5(1), Z1-Z6(2), Z3-Z4(2), Z2-Z4(3), Z3-Z5(3), Z1-Z4(4), Z3-Z6(5), Z2-Z6(6), Z1-Z5(7)	Zona2 - Zona5	1	3
4	{Zona1, Zona2, Zona3, Zona5}	Z1-Z6(2), Z3-Z4(2), Z2-Z4(3), Z1-Z4(4), Z3-Z6(5), Z2-Z6(6), Z5-Z4(7)	Zona1 - Zona6	2	5
5	{Zona1, Zona2, Zona3, Zona5, Zona6}	Z6-Z4(2), Z3-Z4(2), Z2-Z4(3), Z1-Z4(4), Z5-Z4(7)	Zona6 - Zona4	2	7

El costo mínimo final es: 7.

Además, la mejor opción para contruir los caminos es haciendo el recorrido:
Zona1 – Zona2, Zona2 – Zona3, Zona2 – Zona5, Zona1 – Zona6 y Zona6 - Zona4

Parte 2

1.

Lenguaje natural

El algoritmo de Prim permite encontrar un árbol de expansión mínimo en un grafo no dirigido, conexo y ponderado.

En este problema, cada zona se representa como un vértice y cada posible camino entre dos zonas se representa como una arista con peso. El peso indica la distancia o costo de construcción del camino.

El algoritmo comienza desde un vértice origen. A partir de ese vértice, mantiene un conjunto de zonas ya visitadas y otro conjunto de zonas todavía no visitadas. En cada paso, busca la arista de menor peso que conecte una zona visitada con una zona no visitada.

Cuando encuentra esa arista, la agrega al árbol solución y también agrega el nuevo vértice al conjunto de visitados. Este proceso se repite hasta que todas las zonas hayan sido incorporadas.

El resultado final es una lista de aristas que conecta todas las zonas sin formar ciclos y con el menor costo total posible.

Precondiciones

El grafo recibido no es null.

El vértice origen no es null.

El vértice origen pertenece al grafo.

El grafo es no dirigido.

El grafo es conexo.

El grafo es ponderado.

Todas las aristas tienen un peso válido.

Existe una operación para obtener todos los vértices del grafo.

Existe una operación para obtener las aristas adyacentes a un vértice.

Se puede obtener el peso de cada arista.

Se puede identificar el otro extremo de una arista a partir de uno de sus vértices.

Postcondiciones

Se devuelve una lista de aristas que forman el árbol de expansión mínimo.

El árbol resultante conecta todos los vértices del grafo.

Si el grafo tiene V vértices, el árbol resultante tiene $V - 1$ aristas.

El árbol resultante no contiene ciclos.

El costo total de las aristas seleccionadas es mínimo.

El grafo original no se modifica.

Si el grafo no es conexo, el algoritmo informa un error.

Pseudocódigo detallado

PRIM(G , origen)

Inicio

si $G = \text{null}$ o origen = null entonces

 error "Parámetros inválidos"

fin si

si origen no pertenece a G entonces

 error "El origen no pertenece al grafo"

fin si

$T \leftarrow$ lista vacía de aristas

visitados $\leftarrow$ conjunto vacío de vértices

noVisitados $\leftarrow$ conjunto con todos los vértices de G

agregar origen a visitados

eliminar origen de noVisitados

mientras noVisitados no esté vacío hacer

 menorArista $\leftarrow$ null


```

menorCosto <- infinito
para cada vértice u en visitados hacer
    para cada arista a adyacente a u hacer
        v <- obtenerOtroExtremo(a, u)
        si v pertenece a noVisitados entonces
            si peso(a) < menorCosto entonces
                menorCosto <- peso(a)
                menorArista <- a
            fin si
        fin si
    fin para
fin para

```

```

si menorArista = null entonces
    error "El grafo no es conexo"
fin si

agregar menorArista a T
nuevoVertice <- obtenerVerticeNoVisitado(menorArista, visitados)
agregar nuevoVertice a visitados
eliminar nuevoVertice de noVisitados

fin mientras

devolver T

```

Fin

Función auxiliar: obtenerOtroExtremo

```
obtenerOtroExtremo(arista, vertice)
```

Inicio

```
si arista.origen = vertice entonces
```

devolver arista.destino

sino

devolver arista.origen

fin si

Fin

Función auxiliar: obtenerVerticeNoVisitado

obtenerVerticeNoVisitado(arista, visitados)

Inicio

si arista.origen pertenece a visitados y arista.destino no pertenece a visitados
entonces

devolver arista.destino

fin si

si arista.destino pertenece a visitados y arista.origen no pertenece a visitados
entonces

devolver arista.origen

fin si

error "La arista no conecta un vértice visitado con uno no visitado"

Fin

3.

Caso de prueba 1: grafo conexo ponderado válido

Lenguaje natural

Se debe probar el algoritmo con un grafo no dirigido, conexo y ponderado. Este es el caso normal de uso del algoritmo de Prim.

El objetivo es verificar que el algoritmo construya un árbol de expansión mínimo que conecte todos los vértices del grafo, sin ciclos y con costo total mínimo.

Pseudocódigo

CasoDePrueba_GrafoConexoValido

Dado:

un grafo no dirigido, conexo y ponderado

y un vértice origen perteneciente al grafo

Cuando:

se ejecuta PRIM(grafo, origen)

Entonces:

el resultado debe ser un árbol de expansión mínimo

el resultado debe conectar todos los vértices del grafo

el resultado no debe contener ciclos

el resultado debe tener $\text{cantidadVertices}(\text{grafo}) - 1$ aristas

el costo total debe ser mínimo

el grafo original no debe modificarse

Fin

Caso de prueba 2: grafo con un solo vértice

Lenguaje natural

Se debe probar el algoritmo con un grafo que tenga un único vértice y ninguna arista.

Este caso permite verificar que el algoritmo maneje correctamente el caso mínimo válido

Como no hay más vértices que conectar, el resultado esperado es un árbol sin aristas.

Pseudocódigo

CasoDePrueba_GrafoConUnSoloVertice

Dado:

- un grafo no dirigido y ponderado
- con un único vértice
- y un origen perteneciente al grafo

Cuando:

- se ejecuta `PRIM(grafo, origen)`

Entonces:

- el resultado debe ser una lista vacía de aristas
- el algoritmo no debe lanzar error
- el grafo original no debe modificarse

Fin

Caso de prueba 3: grafo no conexo

Lenguaje natural

Se debe probar el algoritmo con un grafo no conexo. Este caso es importante porque Prim necesita que todos los vértices estén conectados para poder formar un árbol de expansión.

Si el grafo no es conexo, el algoritmo no podrá alcanzar todos los vértices desde el origen. Por lo tanto, debe detectar esta situación e informar un error.

Pseudocódigo

CasoDePrueba_GrafoNoConexo

Dado:

- un grafo no dirigido y ponderado
- que no es conexo
- y un vértice origen perteneciente al grafo

Cuando:

- se ejecuta `PRIM(grafo, origen)`

Entonces:

- el algoritmo debe detectar que no puede conectar todos los vértices

- el algoritmo debe informar que el grafo no es conexo

- no debe devolverse un árbol de expansión mínimo inválido

Fin

Caso de prueba 4: origen inexistente

Lenguaje natural

Se debe probar qué ocurre cuando el vértice origen no pertenece al grafo. El algoritmo debe validar esta condición antes de comenzar la búsqueda de aristas

Si el origen no existe en el grafo, no es posible iniciar correctamente Prim.

Pseudocódigo

CasoDePrueba_OrigenInexistente

Dado:

- un grafo no dirigido, conexo y ponderado

- y un vértice origen que no pertenece al grafo

Cuando:

- se ejecuta PRIM(grafo, origen)

Entonces:

- el algoritmo debe informar que el origen no pertenece al grafo

- no debe comenzar la construcción del árbol

- el grafo original no debe modificarse

Fin

Caso de prueba 5: parámetros inválidos

Lenguaje natural

Se debe probar que el algoritmo valide correctamente los parámetros recibidos. En particular, debe verificarse qué ocurre si el grafo recibido es nulo o si el origen recibido es nulo.

Estos casos son importantes porque, si no se validan al inicio, la implementación podría fallar durante la ejecución.

Pseudocódigo de especificación

CasoDePrueba_ParametrosInvalidos

Dado:

- un grafo nulo

- o un origen nulo

Cuando:

- se ejecuta PRIM(grafo, origen)

Entonces:

- el algoritmo debe informar que los parámetros son inválidos

- no debe intentar recorrer el grafo

- no debe intentar seleccionar aristas

Fin

Caso de prueba 6: grafo con empates de peso

Lenguaje natural

Se debe probar el algoritmo con un grafo donde existan aristas candidatas con el mismo peso mínimo.

Este caso es importante porque Prim puede tener más de una solución válida. Por eso, el test futuro no debería exigir necesariamente una única combinación exacta de aristas, sino verificar que el resultado sea un árbol de expansión mínimo válido.

Pseudocódigo

CasoDePrueba_GrafoConEmpates

Dado:

un grafo no dirigido, conexo y ponderado

donde existen aristas con pesos iguales

Cuando:

se ejecuta $\text{PRIM}(\text{grafo}, \text{origen})$

Entonces:

el resultado debe conectar todos los vértices

el resultado no debe contener ciclos

el resultado debe tener $\text{cantidadVertices}(\text{grafo}) - 1$ aristas

el costo total debe ser mínimo

no se debe asumir que existe una única solución correcta

Fin

Caso de prueba 7: grafo con varias aristas candidatas

Lenguaje natural

Se debe probar que, en cada paso, el algoritmo elija una arista válida: debe conectar un vértice ya visitado con un vértice no visitado.

Este caso permite verificar que el algoritmo no agregue aristas entre dos vértices ya visitados, ya que eso podría generar ciclos.

Pseudocódigo de especificación

CasoDePrueba_SeleccionCorrectaDeAristas

Dado:

un grafo no dirigido, conexo y ponderado

donde existen varias aristas candidatas en cada paso

Cuando:

se ejecuta PRIM(grafo, origen)

Entonces:

cada arista agregada debe conectar un vértice visitado con uno no visitado

no debe agregarse una arista entre dos vértices ya visitados

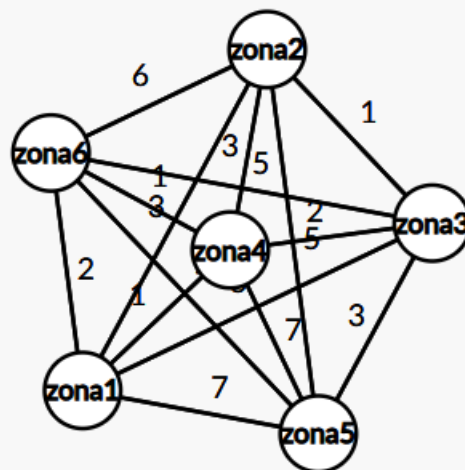
no deben generarse ciclos

al finalizar, todos los vértices deben estar visitados

Fin

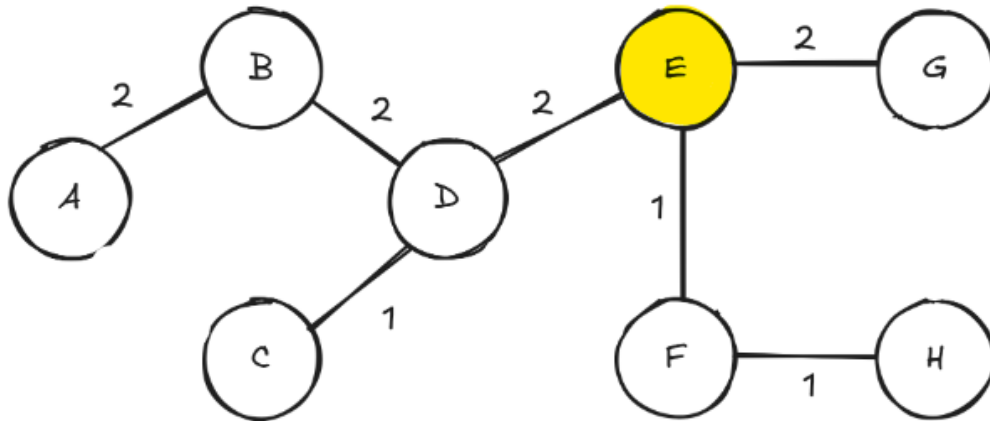
- Ejercicio 9

1.



Ejercicio 10:

Parte 1:



Parte 2:

Lenguaje natural:

Primero se marca el vértice origen como visitado y se lo agrega a una cola de vértices pendientes. Luego, mientras la cola no esté vacía, se extrae el primer vértice de la cola. En ese momento se aplica el Consumer sobre ese vértice. Después se revisan todos sus vértices adyacentes. Por cada adyacente, se verifica si ya fue visitado. Si no fue visitado, se lo marca como visitado y se lo agrega al final de la cola. También puede registrarse la arista que conecta al vértice actual con ese adyacente, ya que esa arista forma parte del árbol abarcador de la búsqueda. El proceso termina cuando la cola queda vacía.

Precondiciones:

- El grafo debe estar inicializado.
- El vértice origen debe existir dentro del grafo.
- La estructura de visitados debe estar inicialmente vacía.
- La cola de pendientes debe estar inicialmente vacía.
- Debe existir una forma de obtener las adyacencias de cada vértice.
- Si se usa un Consumer, este debe estar definido y no ser null.

-

Postcondiciones:

- Todos los vértices alcanzables desde el origen quedan marcados como visitados.
- Cada vértice alcanzable desde el origen es procesado una sola vez.
- La cola de pendientes queda vacía al finalizar.
- No se visitan vértices que no sean alcanzables desde el origen.
- El recorrido respeta el orden de búsqueda en amplitud, es decir, primero se visitan los vértices más cercanos al origen y luego los de mayor distancia.

Pseudocódigo:

BFS(G, origen)

Comienzo

 crear conjunto visitados

 crear cola pendientes

 marcar origen como visitado

 encolar origen en pendientes

 mientras pendientes no esté vacía hacer

 actual ← desencolar pendientes

 accion(actual)

 para cada adyacente de actual hacer

 si adyacente no pertenece a visitados entonces

 marcar adyacente como visitado

 encolar adyacente en pendientes

 fin si

 fin para

fin mientras

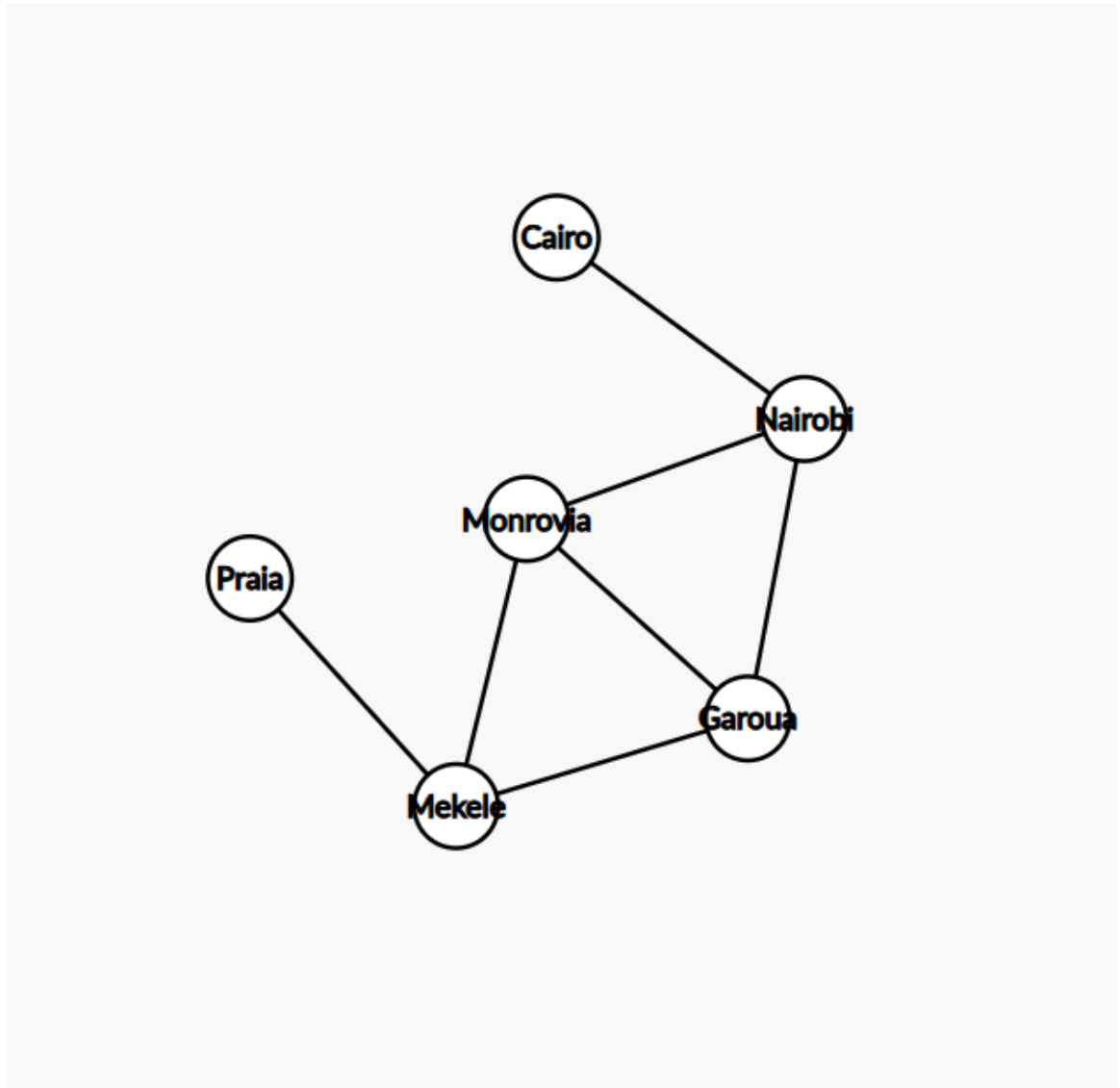
Fin

Orden: $O(V + E)$

Ejercicio 11:

Parte 1:

1.



2. Los puntos de articulación son aquellos que al eliminarse, aíslan nodos del grafo.

Los puntos de articulación son Mekele y Nairobi.

Si falla Mekele, la ciudad de Praia queda desconectada del resto de la red.

Si falla Nairobi, la ciudad de Cairo queda desconectada.

Parte 2:

Lenguaje natural

Para hallar los puntos de articulación de un grafo no dirigido se realiza una búsqueda por profundidad.

Durante el recorrido se guarda, para cada vértice, el momento en que fue descubierto y el menor tiempo de descubrimiento al que puede llegar desde su subárbol. Esto permite saber si existe un camino alternativo que conecte ese subárbol con una parte anterior del recorrido.

Un vértice es punto de articulación si, al eliminarlo junto con sus aristas, alguna parte del grafo queda desconectada del resto.

Se analizan dos casos:

Si el vértice es la raíz del recorrido DFS, será punto de articulación si tiene más de un hijo.

Si el vértice no es raíz, será punto de articulación si alguno de sus hijos no puede llegar a un antecesor del vértice mediante otro camino.

De esta forma, el algoritmo identifica los vértices cuya falla podría afectar la conectividad de la red.

Precondiciones

El grafo recibido no es null.

El grafo es no dirigido.

El grafo es conexo, según lo indicado en el enunciado.

Los vértices del grafo no son null.

Existe una operación para obtener todos los vértices del grafo.

Existe una operación para obtener los adyacentes de un vértice.

Se puede comparar correctamente si dos vértices son iguales.

Postcondiciones

Se devuelve una lista con los puntos de articulación del grafo.

Un vértice aparece en la lista si, al eliminarlo junto con sus aristas, aumenta la cantidad de componentes conexas.

Si el grafo no tiene puntos de articulación, se devuelve una lista vacía.

El grafo original no se modifica.

Cada punto de articulación aparece una sola vez en el resultado.

Pseudocódigo

En la clase vértice

ptoArt (linkedList<vertice> puntos, int[] cont)

Comienzo

cont[0] ++

this.numBp <- cont[0]

this.numBajo <- cont[0]

linkedList<vertice> hijos

Para cada adyacencia en this.adyacencias

adyacente <- adyacencia.destino

Si no(adyacente.visitado) entonces

adyacente.ptosArt(puntos, cont)

hijos.add(adyacente)

this.numBajo <- mínimoEntre (this.numBajo, adyacente.numBajo)

sino

this.numBajo <- mínimoEntre (this.numBajo, adyacente.numBp)

finsi

fin para cada

Si this. numBp > 1 entonces

```
Para cada hijo en hijos hacer
Si hijo.numBajo  $\geq$  this.numBp entonces
puntos.add(this)
fin si
fin para cada
Sino
Si hijos.largo > 1 entonces
puntos.add(this)
fin si
fin si
Fin
```

Ejercicio 12:

Parte 1:

El método numBacon se implementó utilizando la búsqueda en amplitud, ya que el número de Bacon corresponde a la cantidad mínima de vínculos entre un actor y Kevin_Bacon. Cada vínculo se representa como una arista del grafo no dirigido. La búsqueda comienza desde el actor consultado y avanza por niveles hasta encontrar a Kevin_Bacon. El nivel en el que se encuentra corresponde al número de Bacon. Si no existe camino, se devuelve -1.

Tiene complejidad $O(V+E)$

3.

El grafo de actores se representa como un grafo no dirigido, donde cada vértice es un actor y cada arista representa que dos actores participaron en una misma película.

Para calcular el número de Bacon se utiliza búsqueda en amplitud, ya que este recorrido explora el grafo por niveles. Cada nivel representa una distancia en cantidad de relaciones entre actores. Por eso, la primera vez que se encuentra a Kevin_Bacon, la distancia obtenida corresponde al menor número de Bacon posible.

El método devuelve 0 si el actor consultado es Kevin_Bacon. Devuelve un número positivo si existe un camino entre el actor y Kevin_Bacon. Devuelve -1 si el actor no existe, si Kevin_Bacon no está en el grafo o si no existe conexión entre ambos.

Resultados obtenidos:

John_Goodman: 4

Tom_Cruise: 4

Jason_Statham: 4

Lukas_Haas: 5

Djimon_Hounsou: 6

Estos resultados fueron recibidos por consola y guardados en el archivo salida.txt.

Se desarrollaron casos de prueba para verificar: Kevin_Bacon con número 0, actor directamente conectado, actor indirectamente conectado, elección del camino más corto, actor no conectado, actor inexistente y grafo sin Kevin_Bacon.